

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	ANALISIS Y DISEÑO DE ALGORITMOS				
TÍTULO DE LA PRÁCTICA:	Algoritmos: Definición, características, corrección y eficiencia				
NÚMERO DE PRÁCTICA:	1	AÑO LECTIVO:	2025-B	NRO. SEMESTRE:	IV (cuarto)
TIPO DE PRÁCTICA:	INDIVIDUAL	☒			
	GRUPAL	☐			
FECHA INICIO:	08/09/2025	FECHA FIN:	15/09/2025	DURACIÓN:	7 Días
RECURSOS A UTILIZAR: • Un computador. • Material del curso. • Bibliografía del curso					
DOCENTE(s): Diego Alonso Iquira Becerra					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Comprender la definición de algoritmo y su importancia en la resolución de problemas. • Reconocer las características esenciales de un algoritmo bien definido. • Analizar y demostrar la corrección o incorrección de un algoritmo. • Evaluar y comparar la eficiencia de algoritmos. • Aplicar notación asintótica para expresar el costo temporal y espacial.	
TEMAS: • Definición de algoritmo. • Características de los algoritmos. • Eficiencia algorítmica y comparación. • Notación asintótica: O, Ω, Θ.	
COMPETENCIAS	C.a Capacidad de análisis de problemas computacionales.
	C.b Diseño de soluciones algorítmicas eficientes.
	C.c Validación y verificación de algoritmos.
	C.d Comunicación técnica y redacción formal.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1. Definición de Algoritmo

Un algoritmo es una secuencia finita y ordenada de pasos que se ejecutan de manera sistemática para resolver un problema.

- Ejemplo cotidiano: Una receta de cocina es un algoritmo.
- Ejemplo computacional: Calcular el promedio de 3 números.

Código en C++:

```
#include <iostream>
using namespace std;
int main() {
    int a=10, b=20, c=30;
    float promedio = (a+b+c)/3.0;
    cout << "Promedio = " << promedio;
    return 0;
}
```

2. Tipos de problemas que resuelven los algoritmos

Problemas numéricos:

Son aquellos en los que se busca obtener un valor numérico a partir de operaciones matemáticas. Ejemplo, Cálculo del Máximo Común Divisor (MCD), Obtención del mínimo común múltiplo, etc

Aplicaciones prácticas:

- Criptografía (operaciones con números primos grandes).
- Finanzas (cálculo de intereses compuestos).
- Ingeniería (cálculo numérico y simulaciones).

```
#include <iostream>
using namespace std;

int main() {
    int N = 10;

    int suma = 0;
    for (int i = 1; i <= N; i++) {
        suma += i;
    }
    cout << "Suma iterativa de 1 a " << N << " = " << suma << endl;

    int suma_formula = (N * (N + 1)) / 2;
    cout << "Suma con fórmula de 1 a " << N << " = " << suma_formula << endl;

    return 0;
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 3

Problemas de búsqueda:

Consisten en encontrar un elemento dentro de una estructura de datos (lista, árbol, grafo, etc.). Ejemplo, Búsqueda secuencial: recorrer uno a uno los elementos hasta encontrar el valor

Aplicaciones prácticas:

- Bases de datos (encontrar un registro por su clave).
- Motores de búsqueda (Google encuentra documentos relevantes).
- Juegos de estrategia (encontrar el mejor movimiento posible).

Problemas de optimización:

Aquí se busca la mejor solución (mínima o máxima) entre varias posibles, Por ejemplo encontrar la ruta más corta en un mapa (Dijkstra, A*) o el problema del viajero (TSP): hallar la ruta más corta que visita todas las ciudades.

Aplicaciones prácticas:

- GPS y aplicaciones de transporte (Google Maps, Waze).
- Logística y transporte (optimización de rutas de camiones).
- Inteligencia Artificial (planificación de tareas).

Problemas de clasificación:

Consisten en organizar los datos según un criterio (numérico, alfabético, etc.), Por ejemplo Bubble Sort, Selection Sort, Insertion Sort: algoritmos simples de ordenamiento

Aplicaciones prácticas:

- Listas de estudiantes ordenados por nota.
- Sistemas de recomendación (ordenar productos por relevancia).
- Procesamiento de datos masivos (ordenar logs de un servidor).

Problemas de decisión:

Consisten en aquellos en los que la salida es simplemente sí/no (verdadero/falso), Por ejemplo Determinar si un número es primo, Preguntar si existe una ruta entre dos nodos.

Aplicaciones prácticas:

- Seguridad informática (¿es válido un acceso?).
- Procesamiento de imágenes (¿es este píxel parte de un borde?).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 4

3. Características de los algoritmos

- **Finitud:** Debe terminar después de un número finito de pasos.
- **Definición:** Cada paso debe ser claro y sin ambigüedad.
- **Precisión:** Debe estar expresado de manera exacta.
- **Generalidad:** Aplicable a una clase de problemas.
- **Eficiencia:** Uso adecuado de recursos.
- **Correctitud:** Genera resultados esperados para todas las entradas válidas.

4. Evaluación y comparación de la eficiencia de algoritmos

Cuando diseñamos un algoritmo no basta con que sea correcto (que dé la respuesta correcta), también debe ser eficiente, es decir, consumir pocos recursos (tiempo y memoria).

La eficiencia se evalúa principalmente en dos dimensiones:

- **Complejidad temporal (tiempo de ejecución)**
Mide el número de operaciones que realiza un algoritmo en función del tamaño de la entrada n .
Imagina que quieres encontrar un nombre en una lista de alumnos.
 - ✓ **Método 1:** revisar uno por uno desde el principio hasta el final. Si la lista tiene 30 alumnos, en el peor caso revisarás los 30.
 - ✓ **Método 2:** si la lista está ordenada alfabéticamente, puedes abrirla por la mitad y decidir si tu nombre está antes o después. Así reduces mucho las comparaciones.
Ambos métodos encuentran al alumno, pero el segundo lo hace más rápido.
- **Complejidad espacial (uso de memoria)**
Mide la cantidad de memoria extra que necesita un algoritmo para ejecutarse.
Supongamos que quieres ordenar una lista de 10 números.
 - ✓ **Método 1:** comparas e intercambias posiciones directamente en la misma lista. No necesitas memoria extra, solo usas la lista original.
 - ✓ **Método 2:** copias todos los números a otra lista auxiliar para luego volver a ordenarlos. Este método ocupa más memoria porque trabaja con dos listas en lugar de una.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 5

Los dos métodos ordenan los números, pero uno consume más memoria que el otro.

5. Notación asintótica

- Cuando estudiamos algoritmos, no solo nos interesa si funcionan, sino también qué tan rápido crecen en tiempo o en memoria cuando aumentamos el tamaño del problema (por ejemplo, cuando tenemos más datos).
 - La notación asintótica es una forma de describir ese crecimiento sin importar el computador, el lenguaje de programación ni el compilador.
 - Es como decir: *“¿qué pasará con mi algoritmo si en vez de 10 datos tengo 1,000 o 1 millón?”*
- Un algoritmo puede hacer pocas o muchas operaciones.
 - Lo que nos interesa es cómo aumenta ese esfuerzo a medida que la entrada crece.
 - No medimos segundos exactos, sino una estimación del comportamiento general.

Supongamos que tenemos una lista de nombres:

- **Búsqueda uno por uno (lineal):** si hay 10 nombres, revisamos hasta 10; si hay 100, revisamos hasta 100; si hay 1,000, revisamos hasta 1,000.
A medida que la lista crece, el tiempo de búsqueda crece igual de rápido.
- **Búsqueda por mitades (binaria):** si hay 1,000 nombres, no necesitamos revisar todos.
Miramos en la mitad → reducimos a 500 → luego a 250 → hasta encontrar el nombre.
El tiempo crece mucho más lento, incluso aunque aumente la lista.

La notación asintótica nos permite comparar algoritmos de manera justa:

- El tiempo que tarda un programa en segundos puede variar según el computador.
- Pero el número de pasos que hace el algoritmo siempre crece igual sin importar dónde lo ejecutes.

O grande (O) → Peor caso (límite superior): cuánto tarda el algoritmo en la situación más difícil, describe el máximo esfuerzo que puede requerir un algoritmo.

Ejemplo: buscar un nombre que no está en la lista (revisas todo).

Ω grande (Omega) → Mejor caso (límite inferior): cuánto tarda en la situación más favorable, describe el mínimo esfuerzo posible de un algoritmo.

Ejemplo: buscar un nombre que está al inicio de la lista (lo encuentras enseguida).

Θ grande (Theta) → Caso promedio (ajuste exacto): lo que suele tardar normalmente, describe el comportamiento típico o promedio del algoritmo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 6

Ejemplo: buscar un nombre en una posición cualquiera, sin saber si está al inicio, en el medio o al final.

Imagina que tienes que buscar un libro en una biblioteca con 1,000 libros:

- **$O(n)$:** En el peor caso lees los 1,000 libros hasta hallarlo.
- **$\Omega(1)$:** En el mejor caso lo encuentras en el primer intento.
- **$\Theta(n)$:** En promedio revisas unos 500 libros.

La notación asintótica es una herramienta matemática para describir cómo crece el tiempo o memoria que necesita un algoritmo cuando aumentamos los datos de entrada.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejemplo: Búsqueda Secuencial

Se desea verificar si un número entero x se encuentra dentro de un arreglo de enteros.

```
#include <iostream>
using namespace std;

int busquedaSecuencial(int arr[], int n, int clave) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == clave)
            return i; // encontrado
    }
    return -1; // no encontrado
}

int main() {
    int datos[] = { 2, 4, 6, 8, 10 };
    int n = 5;
    int clave = 8;

    int pos = busquedaSecuencial(datos, n, clave);

    if (pos != -1)
        cout << "Elemento encontrado en la posicion " << pos << endl;
    else
        cout << "Elemento no encontrado" << endl;

    return 0;
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

1. Peor caso – O grande (O) → Límite superior

El algoritmo tiene que revisar todos los elementos del arreglo:

- Caso 1: La clave no está en el arreglo.
- Caso 2: La clave está en la última posición.

El número de comparaciones será n .

Peor caso: $O(n)$.

2. Mejor caso – Ω grande (Omega) → Límite inferior

La clave se encuentra en la primera posición del arreglo.

- Solo se hace 1 comparación.
- **Mejor caso: $\Omega(1)$.**

3. Caso promedio – Θ grande (Theta) → Ajuste exacto

- Si la clave puede estar en cualquier posición con igual probabilidad, en promedio se encontrará en la mitad del arreglo.
- Comparaciones = $n/2$.
- Se descarta la constante, entonces: **Caso promedio $\Theta(n)$.**

Ejemplo: Búsqueda Binaria

Se desea implementar un algoritmo que permita determinar si un número entero se encuentra en una lista ordenada de números.

```
#include <iostream>
using namespace std;
```

```
int busquedaBinaria(int arr[], int n, int clave) {
    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        if (arr[medio] == clave) {
            return medio;
        }
        else if (arr[medio] < clave) {
            inicio = medio + 1;
        }
        else {
            fin = medio - 1;
        }
    }
    return -1;
}
```

}

```
int main() {  
    int datos[] = { 2, 4, 6, 8, 10, 12, 14 };  
    int n = 7;  
    int clave = 10;  
    int pos = busquedaBinaria(datos, n, clave);  
  
    if (pos != -1)  
        cout << "Elemento encontrado en la posicion " << pos << endl;  
    else  
        cout << "Elemento no encontrado" << endl;  
    return 0;  
}
```

1. Peor caso – O grande (O) → Límite superior

El peor caso ocurre cuando la clave no está en el arreglo o está en la última posición posible después de dividir varias veces, Cada paso divide el arreglo a la mitad:

- Paso 1: n elementos → se reduce a $n/2$
- Paso 2: $n/2$ → se reduce a $n/4$
- Paso k: $n/(2^k)$

$$n/(2^k)=1 \Rightarrow k=\log_2(n)$$

Peor caso: $O(\log n)$.**2. Mejor caso – Ω grande (Omega) → Límite inferior**

Si la clave está justo en la mitad del arreglo en la primera comparación, se encuentra inmediatamente.

- Solo se hace 1 comparación.
- **Mejor caso: $\Omega(1)$.**

3. Caso promedio – Θ grande (Theta) → Ajuste exacto

En promedio, la clave estará en cualquier posición.

El algoritmo seguirá dividiendo el arreglo por la mitad, el número esperado de comparaciones sigue siendo del orden de $\log_2(n)$.

- **Caso promedio: $\Theta(\log n)$.**

Análisis de eficiencia

Cada iteración reduce el espacio de búsqueda a la mitad.

Para un arreglo de tamaño n , el número de comparaciones necesarias crece mucho más lento que el tamaño de la lista.

En términos simples:

- Si hay 1000 elementos, como máximo se hacen unas 10 comparaciones.
- Si hay 1,000,000 de elementos, bastan unas 20 comparaciones.

Criterio	Búsqueda Secuencial	Búsqueda Binaria
Requisito del arreglo	Puede ser ordenado o desordenado.	El arreglo debe estar ordenado previamente.
Idea básica	Revisa los elementos uno por uno hasta encontrar la clave o llegar al final.	Divide el arreglo a la mitad en cada paso y busca en la parte correspondiente.
Mejor caso (Ω)	$\Omega(1) \rightarrow$ la clave está en la primera posición.	$\Omega(1) \rightarrow$ la clave está justo en el medio desde la primera comparación.
Peor caso (O)	$O(n) \rightarrow$ la clave no está o está en la última posición.	$O(\log n) \rightarrow$ la clave no está o se encuentra tras varias divisiones.
Caso promedio (Θ)	$\Theta(n) \rightarrow$ en promedio recorre la mitad del arreglo.	$\Theta(\log n) \rightarrow$ en promedio reduce el problema a $\log_2(n)$ pasos.
Número de pasos	Crece de forma lineal con el tamaño de la lista.	Crece de forma mucho más lenta.
Ejemplo con 1,000 datos	Puede necesitar hasta 1,000 comparaciones.	Necesita como máximo 10 comparaciones ($\log_2(1000) = 10$).
Ventajas	Sencilla de implementar, funciona siempre.	Mucho más rápida en listas grandes.
Desventajas	Muy lenta si la lista es muy grande.	Requiere ordenar la lista antes de buscar.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Diseñen algoritmos enfocados en diferentes tipos de problemas, se debe determinar el O (peor caso), Ω (mejor caso) y Θ (promedio) para cada algoritmo.

Problema numérico:

Diseñar un algoritmo que calcule el Máximo Común Divisor (MCD) de dos números enteros mediante dos funciones una que utilice el método de Euclides y otro con bucles, comparen el tiempo

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 10

de ejecución entre ambos. Expliquen ¿Cuántas divisiones como máximo realiza el algoritmo de Euclides para el MCD?.

Problema de búsqueda

Implementar un algoritmo de búsqueda de cadenas dado un texto y una palabra, determinar si la palabra se encuentra dentro del texto, deben comparar este método simple con la búsqueda de palabras en una lista ordenada, analicen qué pasa si la palabra se repite varias veces.

Expliquen ¿cómo crece el tiempo respecto a la longitud del texto?, ¿cómo crece respecto al tamaño de la lista?

Problema de clasificación

Diseñar un algoritmo que ordene una lista de nombres en orden alfabético, comparen el algoritmo de ordenamiento de la biblioteca estándar `sort()`

Expliquen ¿cómo crece el número de comparaciones si hay n nombres?

Problema de decisión

Crear un algoritmo que determine si un número es palíndromo, expliquen ¿Cuántas comparaciones hace el algoritmo en función del número de dígitos d ?

Problema de eficiencia:

- Calcular la suma de los primeros n números naturales de dos formas: con un bucle y con la fórmula $n(n+1)/2$.
- Comparar la cantidad de pasos realizados en cada caso.

IV. CUESTIONARIO

1. ¿Qué diferencia existe entre la corrección parcial y total de un algoritmo?
2. ¿Qué ventajas tiene el análisis con notación asintótica frente a medir el tiempo real de ejecución?
3. Explique con un ejemplo una relación de recurrencia y su solución.
4. ¿Por qué la eficiencia es tan importante en problemas de gran escala?
5. ¿Qué características debe cumplir un buen algoritmo? Mencione al menos tres y justifique con ejemplos.
6. ¿Qué significa que un algoritmo tenga complejidad $O(1)$? Dé un ejemplo sencillo.

7. Si tienes un algoritmo que resuelve un problema correctamente pero tarda demasiado, ¿qué opciones tienes como programador para mejorar su eficiencia?

V. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- ✓ Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms*. MIT Press.
- ✓ Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
- ✓ Brassard, G., & Bratley, P. (1996). *Fundamentals of Algorithmics*. Prentice Hall.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS: Ejercicios propuestos / Trabajos/ Preguntas formuladas / Proyectos / Resolución de casos	INSTRUMENTOS: Rubricas/Lista de cotejo			
CRITERIOS DE EVALUACIÓN				
Criterios	Muy Bueno	Bueno	Regular	Malo
Comprensión conceptual	Explica con claridad y precisión los conceptos, con ejemplos propios y completos.	Explica los conceptos de forma adecuada, con algunos ejemplos.	Explicación incompleta o con errores menores.	No comprende los conceptos o presenta errores graves.
Desarrollo de ejercicios	Implementa algoritmos correctos, justifica la corrección y compara la eficiencia.	Resuelve correctamente la mayoría de los ejercicios, con pocos errores.	Presenta errores importantes en la implementación o análisis.	No resuelve los ejercicios o están incorrectos.
Informe escrito	Presenta un informe ordenado, bien estructurado, con análisis y casos de prueba claros.	Informe claro pero con detalles faltantes en redacción o ejemplos.	Informe incompleto, desordenado o con análisis poco profundo.	No presenta informe o está ininteligible.
Participación y actitud	Participa activamente, colabora y cumple con plazos y responsabilidades.	Participa en la mayoría de actividades y cumple parcialmente.	Participa poco, con escasa colaboración o retrasos frecuentes.	No participa, incumple plazos y no colabora.